56262

Kolmogorov-Arnold Networks 2

On the expressiveness and spectral bias of KANs
ICLR (2025)

Basic Idea of KAN

- **Basic Idea**
  - High-dimensional functions $y = f(x)$ can be decomposed into compositions of 1D functions

output ↑ $y = f(x)$ ← Input

f =

$$f(x) = \sum_q \Phi_q \left(\sum_p \phi_{q,p}(x_p) \right)$$

  - These 1D functions are learnable (e.g., spline)

- **Advantages**
  - Interpretability : Each function can be inspected
  - Accuracy : Better approximation of structured (PDE) functions
  - Better scaling laws : Faster improvement with model size

Training of KAN 1/6

- **Learnable Edge Functions**
  - Each connection between neurons contains a learnable function:

$$\phi(x) = w_b b(x) + w_s \text{spline}(x)$$

where:

- $b(x)$ is a residual base activation (typically SiLU),
- $\text{spline}(x)$ is a B-spline function,
- w_b and w_s are trainable scaling parameters.

$f(x) = sx + b$

$w = \{s, b\}$

$f(x) = \phi_1 \cdot \phi_2 \cdots \phi_m(x)$

spline

  - Instead of learning a single scalar weight, KAN learns the shape of a function.

Training of KAN 2/6

- **B-Spline Parameterization**
  - The spline component is represented as:

$$\text{spline}(x) = \sum_{i}^{n} c_i B_i(x)$$

where:

- $B_i(x)$ are B-spline basis functions,
- c_i are learnable coefficients.

  - Training KAN therefore means optimizing these spline coefficients.

Training of KAN 3/6

- **Forward Propagation**
  - For a neuron in layer $l+1$, the output is computed as:

$$x_{l+1,j} = \sum_i \phi_{l,j,i}(x_{l,i})$$

  - Each incoming signal is transformed by its corresponding learnable function and then summed.

Training of KAN 4/6

- **Backpropagation and Optimization**
  - KANs are fully differentiable and trained using standard gradient-based optimization.
  - The learning procedure is

1. Forward propagation through spline functions
2. Loss computation (e.g., MSE or cross-entropy)
3. Backpropagation through spline coefficients
4. Parameter updates using optimizers such as Adam or LBFGS

Training of KAN 5/6

- **Grid-Based Learning**
  - KANs discretize spline functions using grids.
  - For example, an initial grid may be:

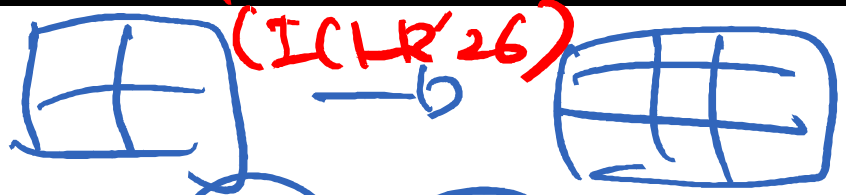
$$[-1, -0.6, -0.2, 0.2, 0.6, 1]$$

  - The spline basis functions are defined over these intervals.
- **Grid Update**
  - During training, activation ranges may change. KAN dynamically updates the spline grid according to the statistics of activations.
  - Then the spline grid is automatically expanded to cover the new range.

Training of KAN 6/6



KANO (ICLR'26)



physic implicit learning → physic discovery

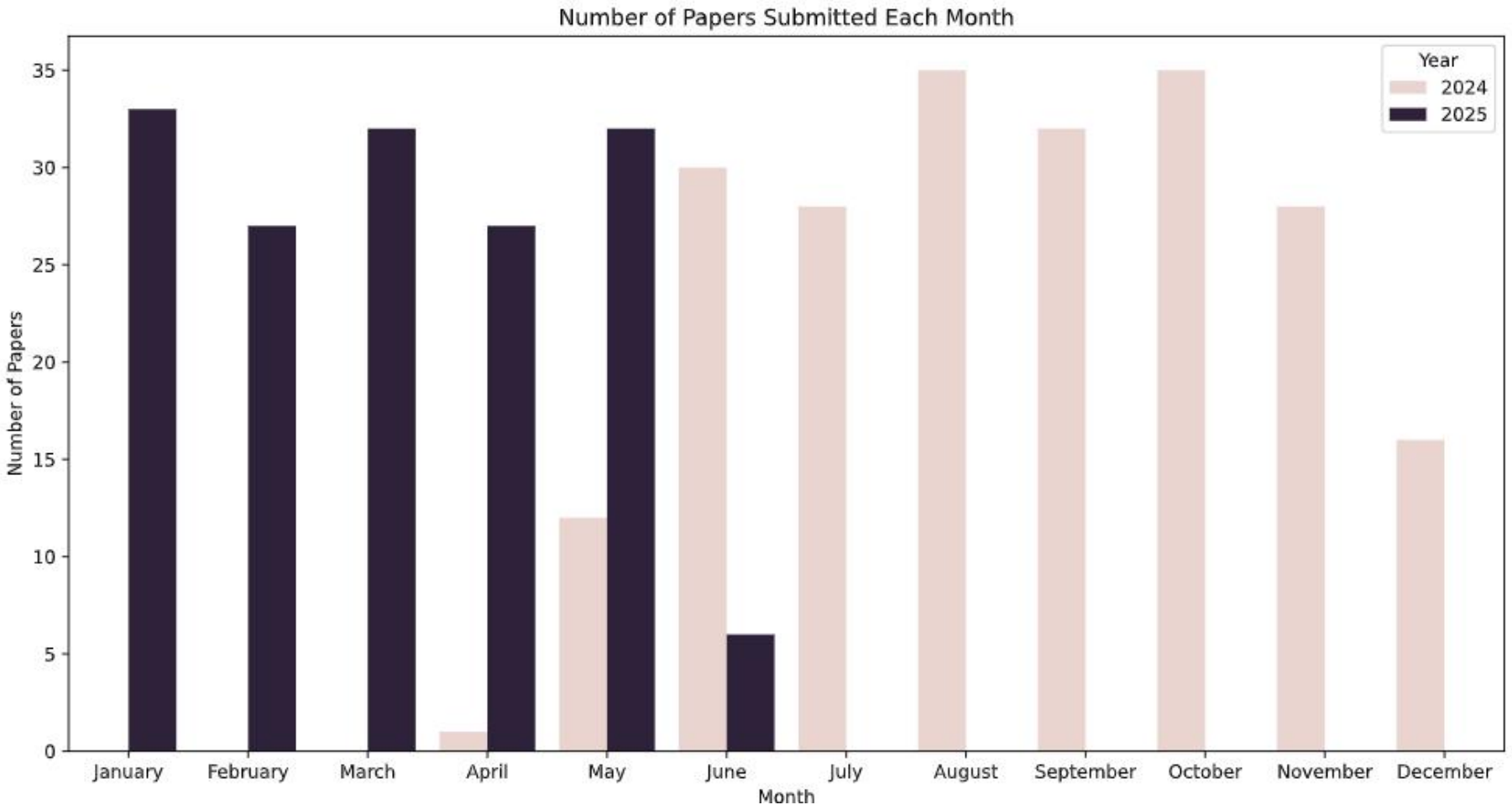
- **Grid Extension**
  - KAN training commonly uses coarse-to-fine refinement.
  - Typical procedure:

$$G = 3 \to 5 \to 10 \to 20$$

  - The network is first trained on a coarse grid and later extended to finer grids without restarting training.
  - This helps avoid poor local minima and improves optimization stability

Recent Variants 1/2

https://ramtinmoslemi.github.io/KAN-Papers



Recent Variants 2/2

https://github.com/mintisan/awesome-kan

- TKAN: Temporal Kolmogorov-Arnold Networks | code | Stars 227
- ReLU-KAN: New Kolmogorov-Arnold Networks that Only Need Matrix Addition, Dot Multiplication, and ReLU | code | Stars 101
- U-KAN Makes Strong Backbone for Medical Image Segmentation and Generation | code | Stars 535
- Kolmogorov-Arnold Networks (KANs) for Time Series Analysis
- Wav-KAN: Wavelet Kolmogorov-Arnold Networks
- A First Look at Kolmogorov-Arnold Networks in Surrogate-assisted Evolutionary Algorithms | code | Stars 41
- FourierKAN-GCF: Fourier Kolmogorov-Arnold Network--An Effective and Efficient Feature Transformation for Graph Collaborative Filtering | code | Stars 41
- A Temporal Kolmogorov-Arnold Transformer for Time Series Forecasting | code | Stars 89
- fKAN: Fractional Kolmogorov-Arnold Networks with trainable Jacobi basis functions | code | Stars 32
- BSRBF-KAN: A combination of B-splines and Radial Basic Functions in Kolmogorov-Arnold Networks | code | Stars 29
- GraphKAN: Enhancing Feature Extraction with Graph Kolmogorov Arnold Networks | code | Stars 71
- rKAN: Rational Kolmogorov-Arnold Networks | code | Stars 21
- SigKAN: Signature-Weighted Kolmogorov-Arnold Networks for Time Series | code | Stars 50
- Demonstrating the Efficacy of Kolmogorov-Arnold Networks in Vision Tasks | code | Stars 11
- KANQAS: Kolmogorov-Arnold Network for Quantum Architecture Search | code | Stars 20
- DeepOKAN: Deep Operator Network Based on Kolmogorov Arnold Networks for Mechanics Problems

Theoretical comparison of KANs and MLPs

- **Why might KANs have advantages over MLPs?**
  1. Expressiveness of KANs : KANs with a large grid size may be more efficient than MLPs at approximating certain functions.
  2. Spectral Bias : KANs are less biased toward low frequencies than MLPs.

$$f(x) = \sin(x) + 0.1\sin(50x)$$

low frequency

high frequency

KAN Architecture

$$f(\mathbf{x}) = f(x_1, \cdots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

1D function

- Each activation function is parameterized as:

$$\phi(x) = w_b \frac{x}{1+e^{-x}} + \sum_{i=0}^{G+k-1} c_i B_i(x)$$

spline

activation

- $B_i(x)$ are B-spline basis functions,
- c_i are trainable spline coefficients,
- w_b is a trainable coefficient for the SiLU activation term,
- G is the spline grid size.

Theorem 3.1

- **How accurately a KAN with spline-based activations can approximate a target function as the spline grid becomes finer**
  - The theorem then states that there exist spline-based approximations: $\Phi_{l,i,j}^{G}$ such that the approximation error decreases as the spline grid size G increases
  - The error bound is:

upper bound

$$0 \leq m \leq k$$

$$\|f - (\Phi_{L-1}^{G} \circ \Phi_{L-2}^{G} \circ \cdots \circ \Phi_{1}^{G} \circ \Phi_{0}^{G})x\|_{C^m} \leq C G^{-k-1+m}$$

target f　　KAN f

(1) G: **Grid Size** — Large G : finer spline representation

(2) k: **Spline Order** — Higher k: smoother spline functions

(3) C^m**-Norm** — measures not only function error but also derivative error up to order m.

Theorem 3.2

- **Any MLP can be represented by a KAN**
  - The theorem then states that

Any MLP using the activation

$$\sigma_k(x) = \max(0, x)^k$$

can be represented exactly by a KAN with:

- "comparable width,"
- "at most twice the depth,"
- "and spline grid size $G = 2$."

  - Each MLP layer can be rewritten as two KAN layers
    - one KAN layer for the linear part,
    - one KAN layer for the activation part.

Theorem 3.3 1/2

- **Any KAN can also be represented by an MLP**
  - The theorem assumes:
    - a KAN with:
      - width W,
      - depth L,
      - spline grid size G,
      - spline order k,

    and additionally assumes: $w_b = 0$

  - Then the theorem states:

    There exists an MLP using $\sigma_k(x) = \max(0, x)^k$

    - width: $(G + 2k + 1)W^2$
    - depth at most: $2L$

Theorem 3.3 2/2

- **Parameter Complexity Comparison**
  - KAN parameter count scales like: $O(GW^2L)$
  - Equivalent MLP parameter count scales like: $O(G^2W^4L)$

KANs may represent certain functions much more efficiently than MLPs.

Especially:

- when grid size is large,
- when functions contain localized high-frequency structures.

Theorem 4.1 1/2

- **Why shallow KANs do not suffer from strong spectral bias in the same way as MLPs**
  - The theorem studies a single-layer KAN
    - one KAN layer,
    - no SiLU residual term,
    - least-squares regression.

$$KAN(x,\theta)_i = \sum_{j=1}^{d} \sum_{l=1}^{G+k-1} c_{ijl} B_l(x_j)$$

  - Least-Squares Loss

$$L(\theta) = \int_\Omega \|f(x) - KAN(x,\theta)\|^2 dx$$

$$L(\theta) = \frac{1}{2}\theta^T M \theta + b^T \theta$$

Hessian matrix

Theorem 4.1 2/2

- **The theorem analyzes the eigenvalues of the Hessian matrix**
  - It proves that:

Largest eigenvalue

Effective condition number of the Hessian

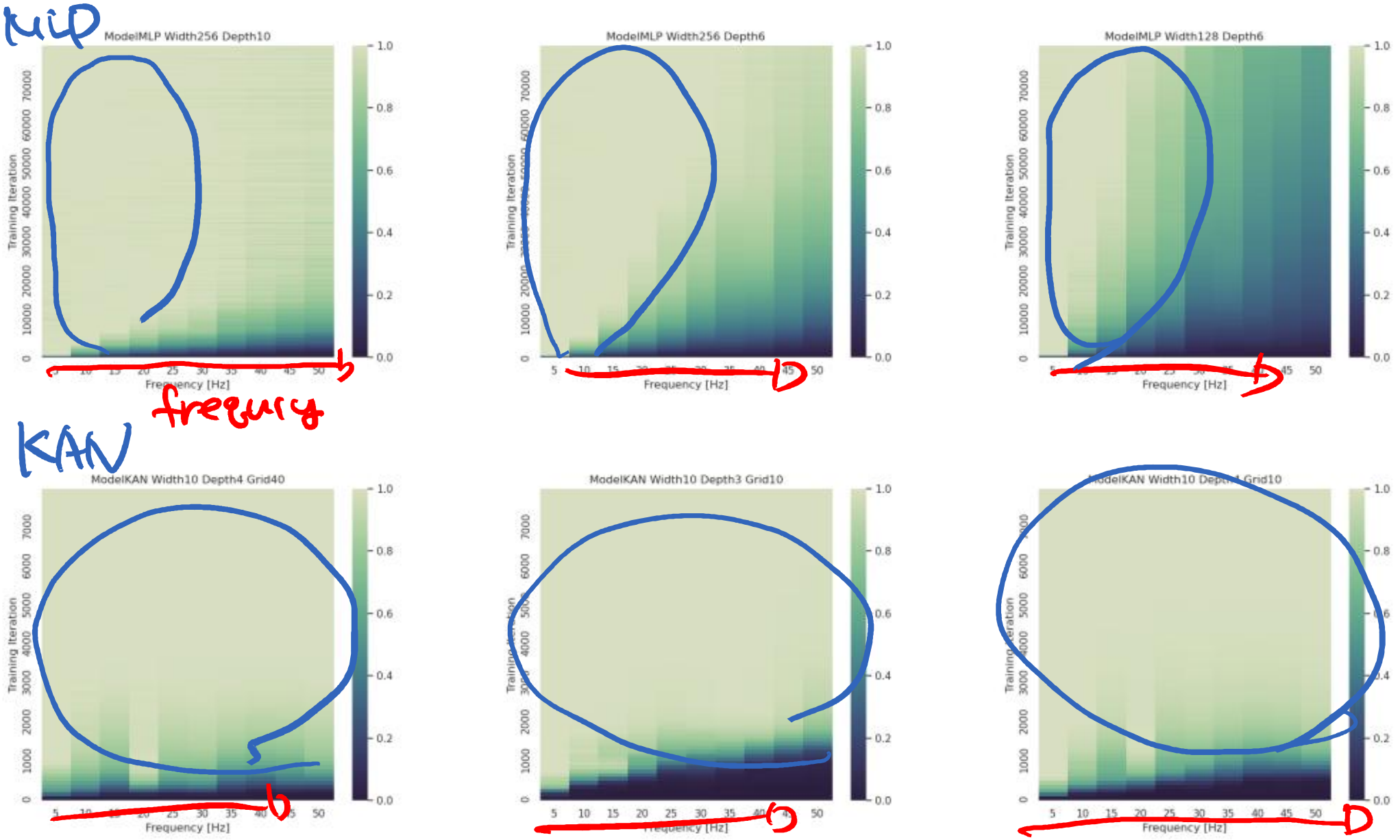
$$\frac{\lambda_N(M)}{\lambda_{d'(d-1)+1}(M)} \leq Cd$$

First meaningful eigenvalue

- $\lambda_i(M)$: eigenvalues of the Hessian,
- d: input dimension,
- d': output dimension,
- C: constant depending only on spline order k.

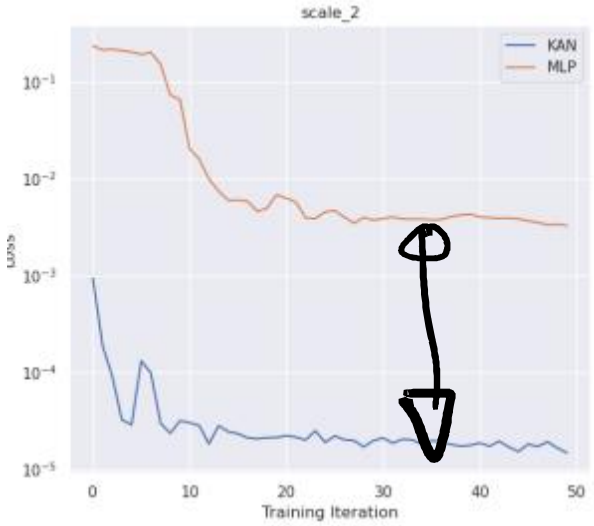
The condition number measures how balanced optimization is.
→ The Hessian is well-conditioned.
→ Most directions learn at similar speeds → Optimization is stable
→ High-frequency components can be learned efficiently.

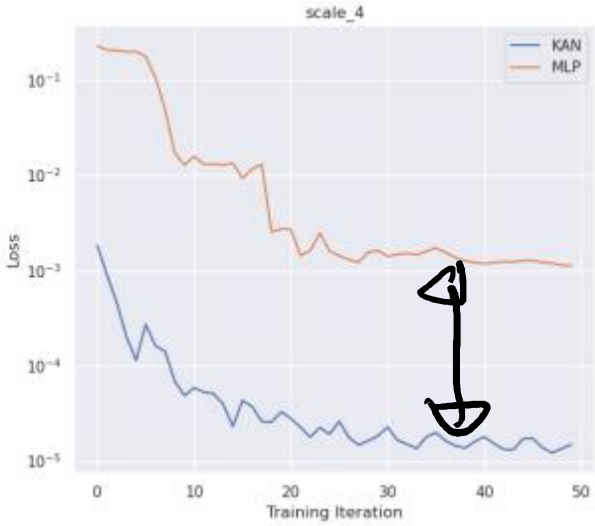
Experiments: Spectral Bias

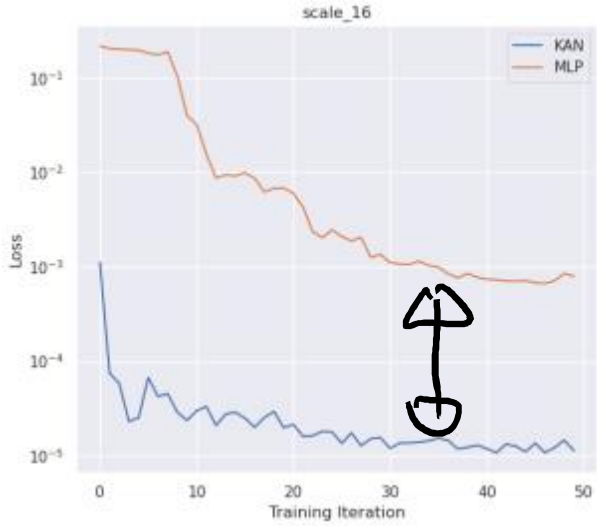


2D Poisson Equation with a High-Frequency Components 1/2

$$-u_{xx} - u_{yy} = f \quad \text{in} [0,1]^2, \quad u(x,0) = u(x,1) = u(0,y) = u(1,y) = 0.$$







2D Poisson Equation with a High-Frequency Components 2/2

$$-u_{xx} - u_{yy} = f \quad \text{in} [0,1]^2, \quad u(x,0) = u(x,1) = u(0,y) = u(1,y) = 0.$$

